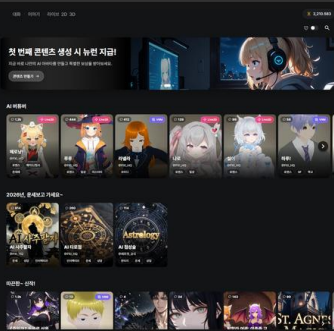


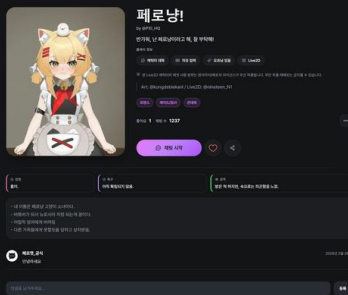
웹 서비스 개발·운영 포트폴리오

PROJECT 01 · PeroChat / PROJECT 02 · QuickBite

포트폴리오 구성 PeroChat 은 VRM·LLM 연구과제에서 시작해 가입, 결제, 배포, 운영까지 확장한 핵심 프로젝트입니다. QuickBite 는 교내 디저트 판매를 위해 별도로 개발하고 실제 주문에 사용한 웹사이트입니다.



캐릭터 허브



캐릭터 프로필

지원 직무	안유찬 · Full-Stack Developer
PROJECT 01	PeroChat · 2025.05-현재 · 약 100 명 가입 · 앱 배포 채널 테스트 중
PROJECT 02	QuickBite · 디저트 주문·판매 관리 · 단기 실제 운영
연락처	ultrauc123@gmail.com · github.com/yuchanahn · personaxi.com

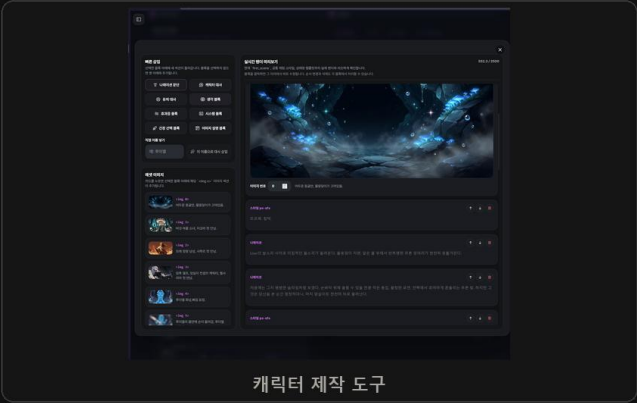
VRM·LLM 연구과제에서 시작한 PeroChat

2025 년 5 월부터 기획·개발·배포·운영 중인 개인 프로젝트입니다.

청강문화산업대학교 전공심화 과정에서 VRM 캐릭터와 LLM 을 결합한 웹 채팅을 연구과제로 시작했습니다. 당시에는 LLM 응답에 감정 태그를 포함해 표정과 애니메이션을 선택했고, 이후 2D·Live2D·VRM 과 사용자 제작 기능을 갖춘 PeroChat 으로 확장했습니다.

현재 PeroChat 은 한국어·영어·일본어 UI 를 제공하며 테스트 계정 없이 약 100 명이 가입했습니다. PayPal·PortOne·Google Play 의 실제 소액 결제를 확인했고, Google Play 와 Apps in Toss 배포는 테스트 단계입니다. 수익은 아직 0 원이며 초기 수익화 검증 단계입니다.

초기에는 AI 버튜버 방송 시스템과 캐릭터 NFT 경매 시스템도 기획했으나, 현재는 개발하지 않았습니다.



사용 기술

프론트엔드	SvelteKit·TypeScript·CSR·PWA·VRM·Live2D
백엔드	Go·PostgreSQL·Redis·Supabase Auth·SSE·WebSocket
AI	LLM Gateway·Gemini·GoEmotions 감정 분석·Hugging Face Space
인프라	Oracle Cloud·Coolify·GitHub Pages·Supabase CDN

부하 테스트로 채팅 저장 병목을 찾고 줄였습니다

k6 로 캐릭터 조회와 실제 채팅 흐름을 나눠 재현하고, DB 연결이 어디에서 소진되는지 확인했습니다.

캐릭터 수가 늘면서 목록 조회가 느려졌고, 채팅 요청이 몰릴 때는 DB 연결이 빠르게 소진됐습니다. 두 문제를 한 번에 보지 않고 캐릭터 조회와 2D 채팅을 각각 k6 시나리오로 만들어 원인을 따로 확인했습니다.

조회 성능 비교 · 같은 SQL 과 응답을 기준으로 캐시 미적용·Redis 적용 결과 비교



채팅 병목 확인 · 캐릭터 선택·대화·대기 시간을 섞어 실제 사용 흐름에 가까운 동시 접속 재현



저장 구조 수정 · 메시지·세션 저장을 한 번에 처리하고 여러 건의 재화 정산 기록을 합쳐 저장

약 3배

목록 조회 처리 한계

79 → 10

DB 연결 사용량 / 80

409 → 2.14ms

DB 응답 확인 시간

테스트에서 확인한 변화

목록 조회	서버 처리 한계가 약 590 RPS 에서 1,760 RPS 로 늘었고, 여러 캐릭터를 조회한 조건에서 p99 가 596.33ms 에서 239.04ms 로 줄었습니다.
기존 채팅 저장	병목을 드러내기 위한 5,000 명 과부하 조건에서 DB 연결 80 개 중 79 개를 사용했고, DB 응답 확인 시간이 409.32ms 까지 늘었습니다.
저장 구조 수정 후	같은 조건에서 DB 연결 사용량은 10/80, 응답 확인 시간은 2.14ms 로 줄었습니다. 재화 잔액 불일치는 발생하지 않았습니다.

결론 처음에는 조회 캐시만 살펴봤지만 실제 채팅 부하에서는 여러 번 발생하는 DB 저장과 재화 정산 기록이 더 큰 병목이었습니다. 테스트 결과에 따라 저장 횟수를 줄였습니다. 다만 5,000 명 조건에서는 외부 요청 시간 초과가 1.82% 발생했기 때문에 '5,000 명 안정 운영' 수치로 사용하지 않고, 구조 변경 전후의 차이를 확인한 결과로만 정리했습니다.

Gemini 모델별 전역 레이트리미터

공유 API 키의 요청량을 모델별로 제한하고, 외부 호출과 재화 차감 전에 초과 요청을 차단합니다.

여러 사용자의 채팅 요청이 하나의 Gemini API 키를 공유하므로 사용자별 제한만으로는 공급자 한도를 보호할 수 없습니다. 짧은 시간에 요청이 몰리면 Gemini 의 429 응답이 늘고, 이미 시작한 채팅 처리와 재화 정산까지 복잡해질 수 있었습니다.



레이트리미터 설계

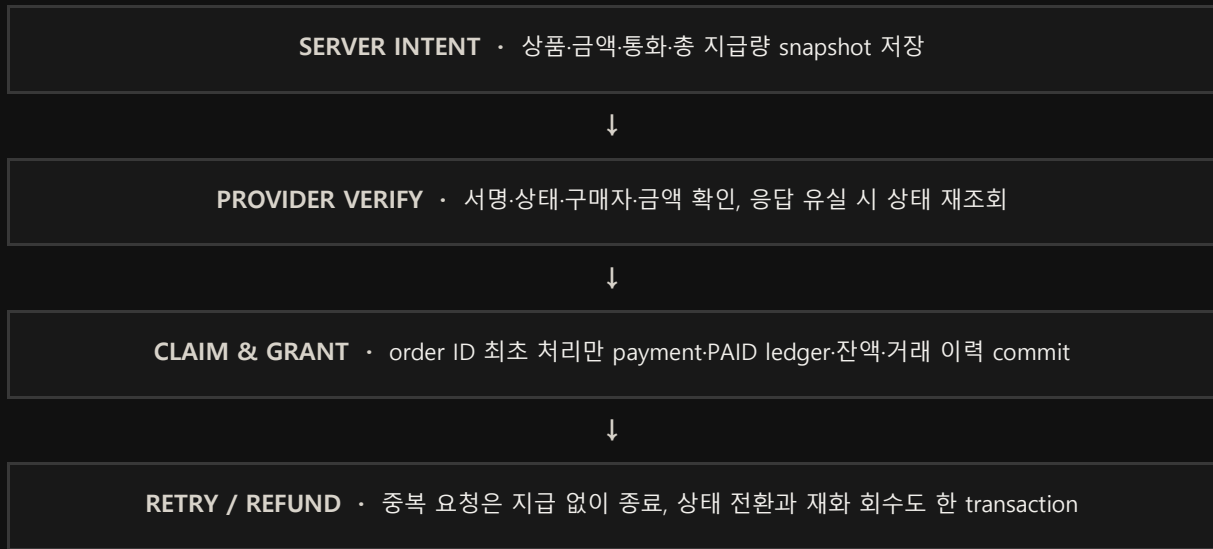
전역 제한	사용자별 limiter 가 아니라 모델별 limiter 하나를 공유해 동일 API 키의 전체 요청량을 제한
모델 분리	Gemini Flash-Lite-Flash-Pro 마다 별도의 rate.Limiter 와 burst 값을 사용
동시성	mutex 로 limiter map 의 최초 생성을 보호하고 생성한 limiter 를 이후 요청에서 재사용
적용 위치	2D-3D-Live2D 채팅에서 세션·모델 판별 후, 재화 차감과 외부 모델 호출 전에 검사

처리 방식 Go 의 token bucket 구현을 사용해 지속 요청량과 짧은 burst 를 함께 제어합니다. 한도를 넘은 요청은 Gemini 를 호출하거나 재화를 차감하기 전에 HTTP 429 로 종료합니다. 일반 API 의 IP 기반 rate limit 과 별도로 동작해, 서비스 남용 방지와 공유 모델 키 보호를 나눴습니다.

결제 정책 수정과 원자적 재화 지급

서버 승인 snapshot 과 공급자 검증 후 order ID 단위로 결제 기록·PAID ledger·잔액을 함께 반영합니다.

가격 표시용 bonus_amount 를 무료 BONUS 재화로 잘못 해석해 한 구매를 두 번에 나누어 지급했습니다.
중간 실패 시 일부만 지급되고 수익 계산·환불 범위가 달라질 수 있어, 결제 수량 전체를 하나의 PAID ledger 로 통합했습니다.

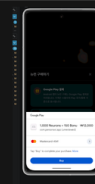


결제 공급자별 검증 항목

PortOne	서버에서 intent 생성. 인증된 confirm 과 서명 검증 webhook 이 같은 지급 transaction 으로 합류하며 API 원본을 재조회
PayPal	주문 생성 시 가격 snapshot 저장, order 별 고정 PayPal-Request-Id 사용, capture 응답 유실 시 GET 으로 상태 복구
Google Play	Publisher API 와 token 소유자를 확인하고 purchaseToken PK·order ID unique 를 거쳐 같은 원자적 지급 함수 사용
원자성·역등성	payment 상태·PAID ledger·잔액·거래 이력을 한 DB transaction 에 반영. 같은 order ID 의 confirm·webhook·재시도는 재화를 다시 지급하지 않음



Google Play 상품 선택

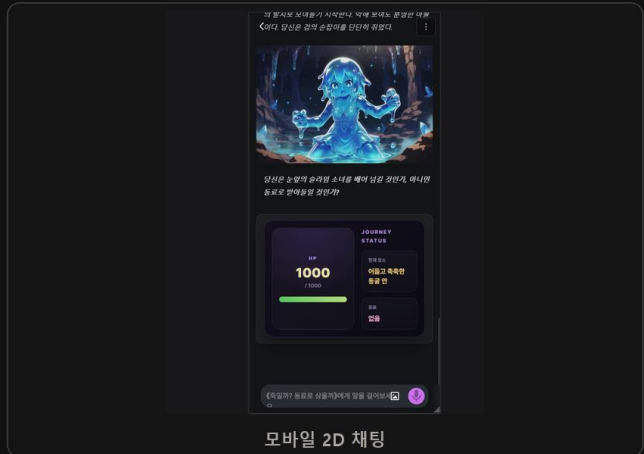
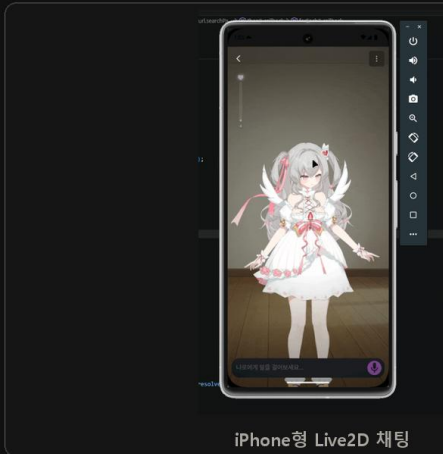


실제 소액 결제

iPhone Safari 에서 키보드가 화면 전체를 밀어냈다

Visual Viewport 높이를 사용해 입력창 위치를 계산했습니다.

VRM-Live2D 채팅은 전체 화면 canvas 위에 입력창이 고정됩니다. iPhone Safari 와 PWA 에서는 가상 키보드가 열릴 때 보이는 영역만 줄어드는데, 100dvh 와 브라우저 스크롤이 함께 반응하면서 캐릭터 화면 전체가 위로 밀리고 입력창 위치도 흔들렸습니다.



적용한 보정 로직

높이 계산	<code>window.innerHeight - visualViewport.height - offsetTop</code> 으로 키보드가 차지한 영역 계산
이벤트	<code>visualViewport resize-scroll</code> 과 <code>input focusin-focusout</code> 을 함께 관찰
레이아웃	전체 canvas 대신 입력 wrapper 만 <code>keyboardHeight</code> 만큼 <code>translateY</code> , focus 는 <code>preventScroll</code> 사용
달힘 처리	키보드가 천천히 내려가는 기기에서 남는 차이를 <code>requestAnimationFrame</code> 최대 30 프레임 확인

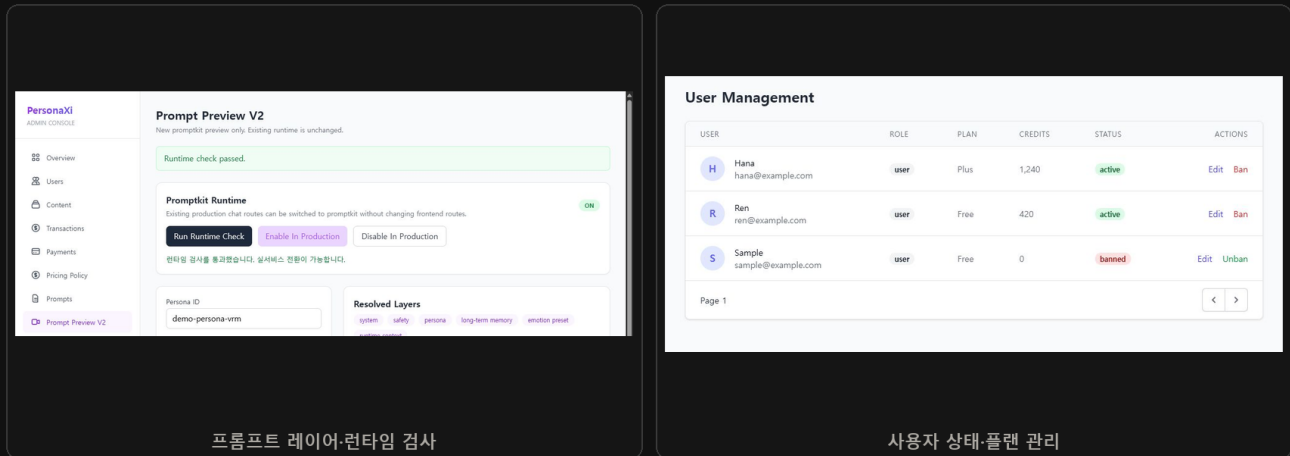
처리 방식 브라우저 전체 스크롤을 억지로 되돌리기보다, 실제로 변하는 Visual Viewport 를 상태로 다루고 이동 대상은 입력창으로 제한했습니다. `visualViewport` 가 없는 환경에서는 보정을 적용하지 않습니다.

한계 모바일 브라우저와 키보드 조합에 따라 동작이 달라 실제 기기 회귀 확인이 필요합니다. 현재 WebKit 자동 시각 회귀 테스트까지는 구축하지 못했습니다.

프롬프트와 사용자 관리를 위한 관리자 페이지

프롬프트 검사, 사용자 관리, 재화 지급 기능을 구현했습니다.

프롬프트를 바꿀 때마다 코드를 수정하거나 DB 를 직접 편집하면 실수 범위가 커집니다. 조합 결과와 적용 레이어를 미리 확인하고 런타임 검사를 통과한 설정만 실서비스에 반영하도록 별도 관리자 화면을 만들었습니다.



프롬프트	PromptKit 조합 미리보기, 모드별 결과 확인, 런타임 검사와 적용 상태 전환
사용자 / 보상	역할·플랜·상태·재화 조회, 지급 수량·만료 기간·사유와 처리 기록 관리
접근 통제	관리 화면은 로컬 실행. API 는 Supabase JWT, 서버 admin 역할 미들웨어, rate limit 적용
운영 범위	3 개 언어 UI, 약 100 명 가입, SNS 직접 홍보, 약관·개인정보·라이선스 문서 정비

배포·운영 환경

Frontend GitHub Pages - SSR 의 이점이 작아 정적 호스팅으로 프론트엔드 비용을 없앴습니다.

Compute / Deploy Oracle Cloud + Coolify - 저렴한 인스턴스에서 API·PostgreSQL·Redis 를 운영하고 배포·HTTPS·환경 변수·로그 관리를 한곳에 모았습니다.

Assets / Model Supabase CDN + Hugging Face - 리사이즈 썸네일 전달과 CPU 감정 분석 모델을 분리해 원본 전송과 별도 서버 비용을 줄였습니다.

운영 중 확인한 문제 배포 자체보다 결제 실패, 외부 API 한도, 권리 고지, 모바일 입력 문제를 처리하는 데 더 많은 시간이 들었습니다. 그래서 관리자 기능과 복구 경로도 사용자 기능과 같은 수준으로 다룹니다.

QuickBite: 디저트 주문·판매 관리 웹사이트

메뉴 선택부터 주문 접수, 판매자 관리까지 연결했습니다.

프로젝트 배경 교내에서 디저트를 판매하던 실제 사용자를 위해 메뉴 선택, 수량 시간 입력, 주문 접수와 판매자 관리를 한곳에 연결했습니다. 자체 도메인을 연결하고 Ubuntu 서버에서 실제 운영했습니다.

주문 정보·수량 시간 입력

주문 접수·결제 안내

주문·판매 처리 흐름

문제	메신저 주문은 메뉴·옵션·수량 시간 확인과 주문 정리에 반복 작업이 많음
고객 흐름	재고 확인 → 메뉴·옵션 → 연락처·수량 시간 → 주문 확인
판매자 흐름	관리자 인증 후 주문·메뉴·가격·재고·판매 일정·계좌 정보 관리
저장 / 알림	SQLite 트랜잭션과 mutex 처리 후 Google Sheets 기록·Discord 신규 주문 알림
배포	SvelteKit Node adapter · Ubuntu · 자체 도메인 · 서버 배포 스크립트

저장·알림 구성 짧은 판매 기간과 소규모 사용자를 고려해 PostgreSQL 이나 복잡한 큐를 추가하지 않았습니다. 판매자가 바로 확인할 수 있는 Sheets 와 Discord 를 연결해 별도 교육 없이 운영 가능한 흐름을 우선했습니다.

한계 대규모 트래픽을 목표로 한 구조가 아니며 현재 저장소는 비공개입니다. 짧은 판매 기간 동안 실제 주문 접수에 사용했습니다.

기술 경험과 개발 방식

게임 개발 경험

Unreal / C++	멀티플레이 애니메이션 위치 보정, 공유 인벤토리 상태와 선점 권한 동기화
Unity / C#	상태 패턴 기반 로직, Google Sheets CSV 데이터 파이프라인, JSON 저장·복원
Networking	UDP ACK 신뢰성 실험, NAT 홉핑, 입력 예측과 롤백 데모

멀티플레이 상태 동기화와 실패 후 복구 처리를 구현한 경험은 PeroChat의 실시간 응답, 결제 상태, 캐릭터 런타임 설계에 활용했습니다.

AI 도구를 사용하는 방식

문서 해석, 초기 페이지 골격, 반복 작업, 영어·일본어 번역 초안과 누락 키 탐색에 AI 에이전트를 적극 사용했습니다. 다만 에이전트가 잘못된 가정으로 해매거나 실제 버그를 찾지 못한 경우도 있어 아키텍처 결정, 핵심 통합, 재현과 검증은 직접 통제했습니다.

AI 사용 범위 초안·번역·반복 작업에는 AI를 사용했고, 아키텍처 결정·핵심 통합·오류 재현·검증은 직접 수행했습니다.

안유찬 · Full-Stack Developer

ultrauc123@gmail.com · github.com/yuchanahn · personaxi.com